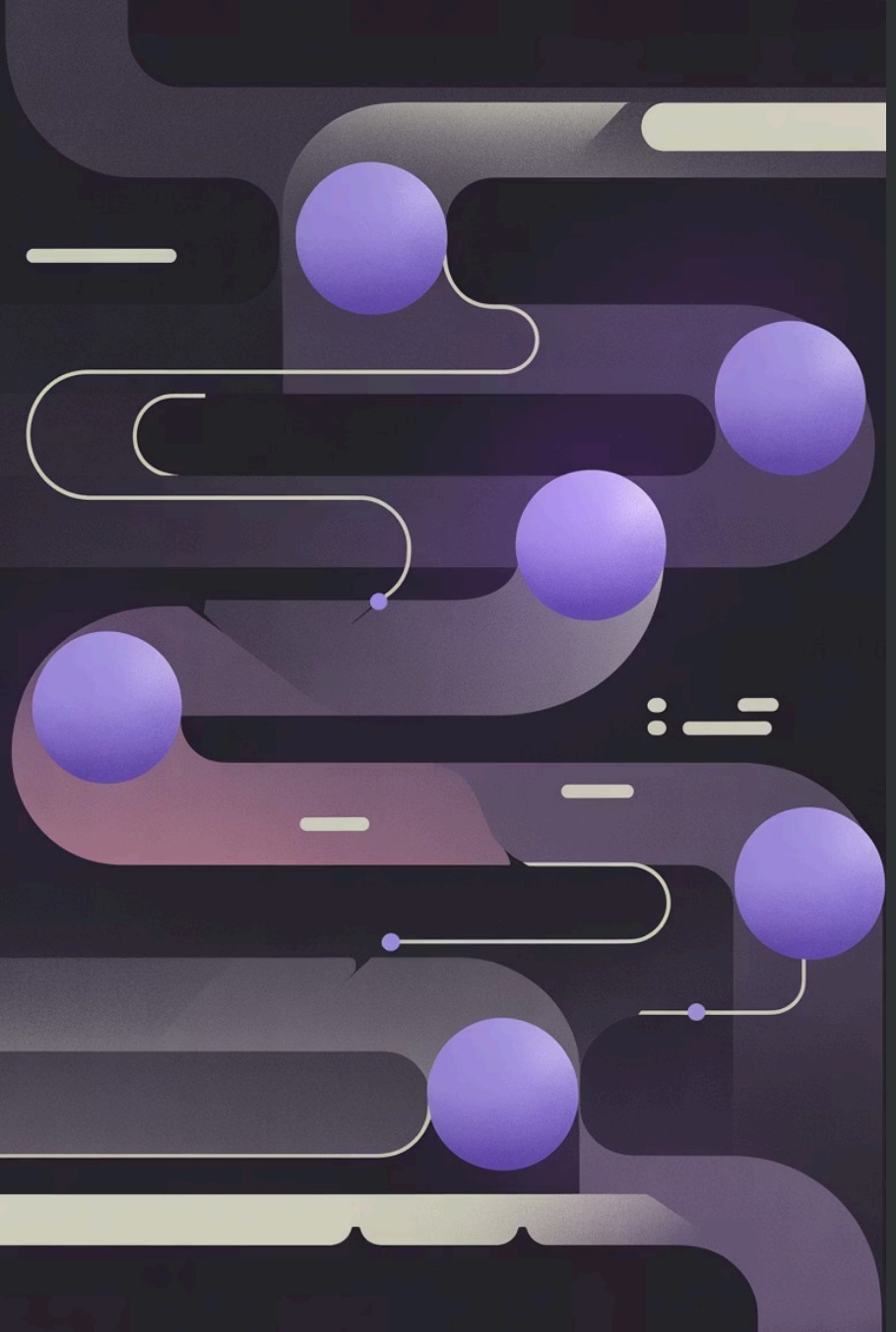

Results and Reflections

A technical summary of Pillar 1 of the Failure Scenario Generation System: architecture decisions, MVP results, deferred scope, and open questions.

ARCHITECTURE

What Was Built

A multi-stage CLI pipeline: **GitHub API → parse → normalise → deduplicate → quality score → store**, with a separate **enrich** command for LLM classification.

Two-Model Data Flow

RawIncident → **IncidentRecord** (Pydantic). Invalid records logged and skipped. Garbage content never reaches storage.

Separate Enrichment Command

LLM enrichment decoupled from ingestion. Runs incrementally or with `--force`; failures don't block new records.

LangChain Abstraction

`.with_structured_output()` enforces typed Pydantic responses. Model swappable via config.

Config-Driven Taxonomy

Three-level taxonomy in `config.yaml`, injected at call time. Editable without code changes, enabling prompt caching.

Git-Object-Style Storage

JSON files in `output/incidents/ab/` subdirectories by ID prefix. Atomic writes via `tempfile + os.replace`.

SHA-Based Change Detection

Previous SHA cached in `run_state.json`. Unchanged source = skip crawl entirely, re-runs under one second.

MVP RESULTS

Pipeline Output: danluu/post-mortems

~150

Incidents Parsed

Raw records from the README

~120

Eligible for Enrichment

After filtering invalid URLs and low-confidence parses

<5s

Full Run Time

Complete pipeline without enrichment

~$1.50

Estimated Corpus Cost

Full enrichment run, 15-20 min wall time

Before Enrichment

Quality scores cluster **0.3–0.6** — terse index entries, not detailed reports.

After Enrichment

Each record gains a structured LLM summary, refined root causes, severity classification, duration, and a three-level taxonomy label with justification. Scores shift upward as completeness and specificity improve.

Pre- and Post-Enrichment Example

File: **output/incidents/ea/eaa38fc3.json**


```
{
  "id": "eaa38fc3",
  "source_url": "https://allegro.tech/2018/08/postmortem-why-allegro-went-down.html",
  "source_type": "POSTMORTEM",
  "source_sha": "3e279cdee234b881f2554cc53f13da2f8440fa3a",
  "title": "Allegro",
  "description": "Allegro. E-commerce site went down after a sudden traffic spike caused by a marketing campaign. The o
utage was caused by a configuration error in cluster resource management which prevented more service instances from st
arting even though hardware resources were available.",
  "company": "Allegro",
  "section": "config_errors",
  "date": null,
  "affected_services": [],
  "root_causes_raw": [
    "E-commerce site went down after a sudden traffic spike caused by a marketing campaign.",
    "The outage was caused by a configuration error in cluster resource management which prevented more service instanc
es from starting even though hardware resources were available."
  ],
  "remediation_actions_raw": [],
  "duration_minutes": null,
  "severity_raw": "outage, down",
  "tags": [
    "config_errors"
  ],
  "quality_score": 0.379,
  "parse_confidence": 0.8,
  "low_quality": false,
  "content_hash": "9a46b25b",
  "potential_duplicate_of": null,
  "duplicate_confidence": null,
  "ingested_at": "2026-04-13T13:05:58.177668+00:00",
  "llm_summary": null,
  "llm_enriched": false,
  "llm_enriched_at": null,
  "taxonomy_category": null,
  "taxonomy_subcategory": null,
  "taxonomy_type": null,
  "taxonomy_justification": null
}
```


Pre-enrichment

File: **output/incidents/ea/eaa38fc3.json**


```
{
  "id": "eaa38fc3",
  "source_url": "https://allegro.tech/2018/08/postmortem-why-allegro-went-down.html",
  "source_type": "POSTMORTEM",
  "source_sha": "3e279cdee234b881f2554cc53f13da2f8440fa3a",
  "title": "Allegro E-commerce Outage — Traffic Spike from Marketing Campaign (2018)",
  "description": "Allegro's e-commerce site went down following a sudden traffic spike caused by a marketing campaign.
The outage was triggered by a configuration error in cluster resource management that prevented additional service inst
ances from starting, even though sufficient hardware resources were available. The incident was resolved by correcting
the misconfiguration to allow the cluster to scale out and handle the increased load.",
  "company": "Allegro",
  "section": "config_errors",
  "date": "2018-08-01",
  "affected_services": [
    "Allegro e-commerce site",
    "Cluster resource management system"
  ],
  "root_causes_raw": [
    "Configuration error in cluster resource management that prevented new service instances from starting despite avai
lable hardware resources",
    "Sudden traffic spike caused by a marketing campaign exceeding the capacity of the running service instances"
  ],
  "remediation_actions_raw": [
    "Corrected the cluster resource management configuration to allow additional service instances to start",
    "Scaled out service instances to handle the increased traffic load"
  ],
  "duration_minutes": null,
  "severity_raw": "complete outage",
  "tags": [
    "config_errors"
  ],
  "quality_score": 0.59,
  "parse_confidence": 0.8,
  "low_quality": false,
  "content_hash": "9a46b25b",
  "potential_duplicate_of": null,
  "duplicate_confidence": null,
  "ingested_at": "2026-04-13T13:05:58.177668+00:00",
  "llm_summary": "Allegro's e-commerce site went down following a sudden traffic spike caused by a marketing campaign.
The outage was triggered by a configuration error in cluster resource management that prevented additional service inst
ances from starting, even though sufficient hardware resources were available. The incident was resolved by correcting
the misconfiguration to allow the cluster to scale out and handle the increased load.",
  "llm_enriched": true,
  "llm_enriched_at": "2026-04-13T13:06:35.640409+00:00",
  "taxonomy_category": "operational",
  "taxonomy_subcategory": "misconfiguration",
  "taxonomy_type": "config_drift",
  "taxonomy_justification": "The outage was caused by a misconfiguration in the cluster resource management system that
 blocked auto-scaling. While a traffic surge was the trigger, the underlying failure was an incorrect configuration tha
t prevented the infrastructure from responding appropriately, making misconfiguration the primary classification."
}
```


Post-enrichment

DEFERRED SCOPE

What Was Not Built

Intentionally Deferred

- **Automatic enrichment**: because the application is local, money is saved by only calling the enrich command when it is needed and on a select number of incidents
- **Concurrent enrichment**: sequential by design; thread pool deferred pending sequential validation
- **Multiple data sources**: parser tightly coupled to danluu README format
- **Database backend**: flat JSON sufficient at 150 records; pgsql planned for scale
- **API layer**: no REST/GraphQL; downstream reads JSON directly
- **Automated scheduling**: manually triggered; no cron or webhook integration

Harder Than Expected

- **Markdown parsing**: inconsistent entry formats required a state machine and careful fixture work
- **HTML extraction**: post-mortem pages vary enormously; truncation partially mitigates noise

Next With Another Week

1. Concurrent enrichment
2. Prompt caching to reduce toke cost
3. Second data source for schema validation
4. Evaluation harness with 50–100 hand-labelled records

OPEN QUESTIONS

Unresolved Risks

LLM Extraction Accuracy

Schema conformance enforced, but factual accuracy unmeasured. No ground-truth benchmark exists.

Parse Confidence Calibration

Heuristic score unvalidated against human judgement; poor calibration degrades the enrichment corpus directly.

Taxonomy Coverage

Defined top-down; real incidents often span categories or fall outside represented types.

Dead Source URLs

Older links unreachable — systematic under-enrichment with no fallback or review flagging.

Quality Score Validity

Completeness and specificity are proxies for usefulness; correlation with downstream performance unknown.

Deduplication Threshold

SimHash at 0.9 set without empirical validation; optimal value likely varies by README section.